```
private readonly ILogger<ProductionEmailService> _logger;
private readonly IConfiguration _config;
private readonly SmtpSettings _settings;                          ❶

public ProductionEmailService(ILogger<ProductionEmailService>
➥ logger, IConfiguration config)
{
    _logger = logger;
    _config = config;
    _settings = _config.GetSection(nameof(SmtpSettings))          ❷
    ➥ .Get<SmtpSettings>();                                       ❷
}
```

❶ Create a field for the SmtpSettings.

❷ Access the SmtpSettings section, and attempt to bind to an instance of the SmtpSettings class.

Within the `SendEmailAsync` method, you use the instance of `SmtpSettings` to provide the values required to configure the `SmtpClient` in a type-safe and cleaner way than was demonstrated using the string-based keys.

**Listing 14.14 Strongly typed configuration**

```
using var smtp = new SmtpClient();
await smtp.ConnectAsync(_settings.Host,_settings.Port, _settings.StartTls);
await smtp.AuthenticateAsync(_settings.Credentials.UserName,
➥ _settings.Credentials.Password);
```

This is not the only way to enjoy strong typing with configuration. We will take a look at binding directly to plain old CLR objects (POCOs) soon, but before that, we will look at another approach that is not dependent on `IConfiguration`, known as the *options pattern*.

## *14.2.4 Using the options pattern*

The options pattern works in a way similar to the previous approach in that it enables working with strongly typed configuration values and relies on the